Appendix: What I added beyond the AI

Tool used throughout: Claude Code (Claude Opus) in plan mode, then agent mode. Full transcripts are in `llm-logs.zip`. Anything below that describes the AI's work can be checked against those logs.

Where AI helped

- **Scoping.** It read the assignment and proposed three candidate problems, each with a one-line architecture idea.
- **Design and build.** It drafted the 5-stage pipeline, the "reproduce the notice, residue must be exactly 0" gate, the rules table, the 12 synthetic cases, the clickable mock, the diagram and the first draft of this brief.
- **Research.** It found volume figures (IRS AUR, CBDT e-campaigns), checked the 1099-K threshold history, and in a second pass confirmed the IRS figure on IRS.gov and flagged India's new Income-tax Act 2025.

Key prompts (verbatim). I steered mostly through choices, not long prompts:

1. `/plan read project.txt and start with a plan`
2. My answers to its scoping questions: **"Tax notice triage"**, **"Clickable mock only"**, and, typed by hand rather than picked from its list, **"india and us"**.
3. `write the appendix, and other things that are remaining`: this started the fact-check pass below.

What I rejected or corrected

- **I rejected all three of its recommendations.** It recommended *loan covenant checks* (marked "Recommended"); I chose tax-notice triage. It recommended a *Python prototype on a free OpenRouter model*; I chose a clickable mock. It offered one jurisdiction or "doesn't matter"; I typed **India and US**. **[Your reason for each, one line: e.g. why notices over covenants.]**
- **An unverified volume figure.** The first draft cited ~987k AUR closures from accounting-news sites and marked it "verify". It is now confirmed on IRS.gov (FY2025: 987,460 cases, $5.9B assessed; FY2024: 1.2M).
- **A blind spot in the brief's own argument.** The brief argues that tax rules decay faster than a model's knowledge, yet the first draft cited only the 1961 Act. India's **Income-tax Act 2025** applies from tax year 2026-27, and commentary maps s.143(1) to **s.270**. This is now noted in §7 as a rules-table version change, not a redesign. That is the design working as intended. *(The mapping is from secondary sources and still needs checking against the enacted text.)*
- **Still unverified, and labelled as such:** 1–3 hours per notice (an assumption), and an Indian notice count (none published).

What came from me

- **Choosing a problem where the input is itself machine output. [Confirm or rewrite in your words: a CP2000 or 143(1)(a) notice comes from a deterministic join, which is what makes "rebuild the notice first" possible. Covenants have no such anchor.]**
- **Two jurisdictions rather than one.** This forced the knowledge out of the model and the code into a per-jurisdiction rules table. If India and the US share one pipeline, adding a third country is a table, not a rewrite (brief §7).
- **Mock over a live prototype.** With canned LLM output and live deterministic checks, the tests isolate the part I'm claiming works (the scaffolding) from the part I haven't proven (extraction on real scans). The brief says this openly.

One thing I'd stand behind without AI

The zero-residue gate, and where it stops working. The notice is the result of a join between third-party reports and the return. So the system first rebuilds the notice's figures from the taxpayer's own copies. It then accepts an explanation only if the records it cites add up to the variance exactly. All the adding is done by code; the model only points at lines. **Where it holds:** in IN-03 the arithmetic closes, but the rules table blocks the draft, because claiming the TDS credit this year contradicts deferring the interest. **Where it breaks:** in US-05 the model cites a real 1040 line with the right amount that already belongs to another fund. The arithmetic passes and the meaning is wrong. An evidence ledger (each source line explains only one amount) fixes reuse, but not a coincidental match. I can walk through both on a whiteboard.